PROYECTO INTERMODULAR II

Desarrollo de Aplicaciones Web

AUTOR:

Aidan Díaz Pérez

TUTORÍA:

Mario Fernández Lago

CONVOCATORIA:

Abril 2026

2VIFC303

CURSO 2025-2026

Índice

INTRODUCCIÓN 3
1. OBJETIVOS DEL PROYECTO 3
1.1. OBJETIVO GENERAL DEL PROYECTO 3
1.2. OBJETIVOS ESPECÍFICOS 4
2. ANÁLISIS Y DISEÑO 5
2.1. REQUISITOS FUNCIONALES 5
2.2. REQUISITOS NO FUNCIONALES 6
2.3. CASOS DE USO 6
2.3.1. DIAGRAMA DE CASOS DE USO 7
2.3.2. DESCRIPCIÓN DE LOS ACTORES 7
2.3.3. DESCRIPCIÓN DE LOS CASOS DE USO 8
2.4. MODELOS DE DATOS 12
2.4.1. DIAGRAMA E/R 12
2.4.2. DIAGRAMA DE CLASES 13
2.5. INTERFACES GRÁFICAS DE USUARIO 14
2.5.1. MAPA DE NAVEGABILIDAD 16
3. PLAN DE PRUEBAS 17
4. TECNOLOGÍAS EMPLEADAS 19
5. PLANIFICACIÓN Y PRESUPUESTO 21
5.1. DIAGRAMA DE GANTT 22
5.2. ESTIMACIONES DE COSTES 22
6. FUTURAS MEJORAS Y AMPLIACIONES 24
6.1. PROPUESTAS DE MEJORA Y AMPLIACIONES 24
6.2. IMPACTO Y RIESGO DE LOS CAMBIOS PROPUESTOS 25
7. RELACIÓN CON LOS MÓDULOS DEL CICLO 26
8. CONCLUSIONES 28
9. DESPLIEGUE DE LA APLICACIÓN 29
9.1. Estructura del proyecto y elección de Spring Boot 29
10. GUÍA DE ESTILOS Y MANUAL DE USUARIO 35
10.1. GUÍA DE ESTILOS 35
10.2. MANUAL DE USUARIO 38
11. WEBGRAFÍA Y FUENTES DE INFORMACIÓN 43

Índice de figuras

Ilustración 1- Diagrama de casos de uso 7
Ilustración 2 - MockUp página principal Ilustración 3 - MockUp login y registrarse 15
Ilustración 4 - MockUp perfil 15
Ilustración 5 - Mapa de navegabilidad 16
Ilustración 6 - Diagrama de Gantt 22
Ilustración 7 -Estructura del proyecto 30
Ilustración 8 - Propiedades del proyecto 31
Ilustración 9 - Código del main 32
Ilustración 10 - Proyecto siendo desplegado 32
Ilustración 11 - Aplicación desplegada 33
Ilustración 12 - Aplicación desplegada Login 33
Ilustración 13 - Aplicación panel administrador 34
Ilustración 14 - Aplicación Panel alumno 34

Índice de tablas

Tabla 1 - Caso de uso (Iniciar sesión) 8
Tabla 2 - Caso de uso (Registrar horas) 8
Tabla 3 - Caso de uso (Validar horas) 9
Tabla 4 - Caso de uso (Subir y gestionar documentos) 9
Tabla 5 - Caso de uso (Evaluar alumno) 10
Tabla 6 - Caso de uso (Generar informes) 10
Tabla 7 - Caso de uso (Gestionar) 10
Tabla 8 - Caso de uso (Editar perfil) 11
Tabla 9 - Caso de uso (Asignar prácticas) 11
Tabla 10 - Tabla de costes 23

INTRODUCCIÓN

El presente documento recoge la documentación correspondiente al Proyecto Intermodular del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web (DAW). Este proyecto tiene como finalidad el diseño y desarrollo de una aplicación web orientada a la gestión de la Formación en Empresa (FE), también conocida como prácticas formativas, realizadas por los estudiantes del ciclo.

En la actualidad, la gestión de las prácticas en empresa en muchos centros educativos se realiza mediante procesos manuales o herramientas poco integradas, como hojas de cálculo, documentos en papel o intercambios de correos electrónicos. Este enfoque genera diversos problemas, como la pérdida de información, la duplicidad de datos, la falta de trazabilidad y una comunicación poco eficiente entre los distintos actores implicados.

Ante esta situación, surge la necesidad de desarrollar una solución tecnológica que permita centralizar toda la información y automatizar los procesos relacionados con las prácticas formativas. La aplicación propuesta busca cubrir esta necesidad mediante una plataforma web que facilite la gestión integral del proceso, desde la asignación de prácticas hasta la evaluación final del alumnado.

El sistema permitirá la interacción de diferentes tipos de usuarios: alumnos, tutores de empresa, tutores del centro educativo y administradores. Cada uno de estos perfiles tendrá acceso a funcionalidades específicas adaptadas a sus necesidades, garantizando así un entorno seguro, organizado y eficiente.

Este documento está estructurado de forma que abarca desde la fase de análisis y diseño hasta el despliegue final de la aplicación, pasando por la planificación, las pruebas realizadas y la relación del proyecto con los contenidos impartidos en el ciclo formativo. Se trata, por tanto, de un documento integral que refleja todo el ciclo de vida del desarrollo software aplicado a un caso de uso real y con impacto directo en el entorno educativo.

1. OBJETIVOS DEL PROYECTO

El proyecto consiste en el desarrollo de una aplicación web destinada a gestionar de forma digital la formación en empresa de los estudiantes. A través de esta plataforma, se pretende centralizar toda la información relacionada con las prácticas, mejorar la organización del proceso y facilitar la interacción entre los distintos usuarios.

La aplicación permitirá gestionar usuarios (alumnos, tutores de empresa, tutores del centro y administradores), empresas, convenios, asignaciones de prácticas, registro de horas, evaluaciones y documentación asociada. Todo ello con el objetivo de sustituir procesos manuales por un sistema automatizado, más eficiente y menos propenso a errores.

1.1. OBJETIVO GENERAL DEL PROYECTO

El objetivo principal de este proyecto es diseñar y desarrollar una aplicación web funcional que permita gestionar de forma integral el proceso de Formación en Empresa (FE) en centros educativos que imparten ciclos formativos de Formación Profesional. La plataforma debe centralizar la información, automatizar procesos y facilitar la comunicación entre todos los actores involucrados: alumnado, tutores del centro, tutores de empresa y personal administrativo.

La solución debe ser accesible, intuitiva y segura, adaptándose a los diferentes roles de usuario y ofreciendo una experiencia de uso coherente tanto en dispositivos de escritorio como en dispositivos móviles.

1.2. OBJETIVOS ESPECÍFICOS

Los objetivos específicos que se derivan del objetivo general son los siguientes:

Gestión de usuarios y roles. Implementar un sistema de autenticación y autorización que permita diferenciar entre los distintos tipos de usuarios (alumno, tutor de centro, tutor de empresa y administrador), garantizando que cada uno acceda únicamente a las funcionalidades que le corresponden.

Registro y validación de horas. Desarrollar un módulo que permita a los alumnos registrar diariamente las horas realizadas en la empresa, y a los tutores validar o rechazar dichos registros de forma sencilla y trazable.

Gestión documental. Incorporar un sistema de subida, almacenamiento y descarga de documentos relacionados con las prácticas (convenios, informes, memorias, etc.), con control de versiones básico y acceso diferenciado por rol.

Comunicación interna. Facilitar la comunicación entre alumnos y tutores mediante un sistema de mensajería interna o notificaciones dentro de la plataforma, reduciendo la dependencia del correo electrónico externo.

Seguimiento y evaluación. Permitir a los tutores del centro realizar el seguimiento del progreso del alumnado y registrar las evaluaciones parciales y finales de forma estructurada.

Generación de informes. Ofrecer la posibilidad de generar informes en formato PDF con los datos más relevantes de cada alumno en prácticas, para su uso administrativo o académico.

Asignación de prácticas. Proporcionar al administrador herramientas para asignar alumnos a empresas y tutores, gestionando la disponibilidad y los convenios de colaboración vigentes.

Despliegue y accesibilidad. Desplegar la aplicación en un entorno de producción accesible desde cualquier navegador web moderno, sin necesidad de instalación por parte del usuario.

2. ANÁLISIS Y DISEÑO

El proyecto se centra en el desarrollo de una aplicación web que permita gestionar de forma eficiente la formación en empresa de los estudiantes. El sistema se diseña teniendo en cuenta la interacción entre diferentes tipos de usuarios, cada uno con funciones específicas dentro del proceso.

La aplicación busca centralizar todas las operaciones relacionadas con las prácticas formativas, evitando la dispersión de información y mejorando la eficiencia del sistema. Para ello, se estructura en distintos módulos funcionales que agrupan las principales características del sistema.

El diseño del sistema tiene en cuenta aspectos fundamentales como la usabilidad, la seguridad, el rendimiento y la accesibilidad, con el objetivo de ofrecer una experiencia de usuario óptima.

2.1. REQUISITOS FUNCIONALES

RF1: Autenticación y autorización basada en roles
El sistema debe permitir a los usuarios autenticarse mediante credenciales seguras y acceder a diferentes funcionalidades en función de su rol. Esto es fundamental para garantizar la seguridad del sistema y evitar accesos no autorizados.

RF2: Gestión de datos (CRUD)
La aplicación deberá permitir realizar operaciones de creación, lectura, actualización y eliminación sobre las entidades principales del sistema, como usuarios, empresas, prácticas y documentos. Esto asegura una gestión completa y flexible de la información.

RF3: Registro y validación de horas
Los alumnos podrán registrar las horas realizadas durante sus prácticas, mientras que los tutores de empresa tendrán la capacidad de validarlas. Este proceso es clave para llevar un control preciso del cumplimiento de las prácticas.

RF4: Subida y gestión de documentos
Los usuarios podrán subir, consultar y descargar documentos relevantes. Esta funcionalidad permite centralizar toda la documentación en un único sistema accesible.

RF5: Evaluación de alumnos
Los tutores podrán evaluar el rendimiento de los alumnos mediante un sistema estructurado, lo que facilita un seguimiento objetivo del desempeño.

RF6: Generación de informes
El sistema permitirá generar informes en distintos formatos, como PDF o Excel, que incluyan información relevante sobre las prácticas.

RF7: Paneles personalizados por rol
Cada tipo de usuario dispondrá de un panel adaptado a sus necesidades, lo que mejora la usabilidad y facilita el acceso a las funcionalidades.

2.2. REQUISITOS NO FUNCIONALES

RNF1: Responsividad
La aplicación debe ser completamente adaptable a distintos dispositivos, garantizando una experiencia de usuario adecuada tanto en ordenadores como en dispositivos móviles.

RNF2: Seguridad
Se implementarán medidas como el cifrado de contraseñas, validación de datos y control de accesos para proteger la información del sistema.

RNF3: Rendimiento
El sistema deberá ofrecer tiempos de respuesta rápidos, evitando esperas innecesarias y mejorando la experiencia del usuario.

RNF4: Disponibilidad
La aplicación deberá estar disponible en todo momento, asegurando el acceso continuo a la información.

2.3. CASOS DE USO

Los casos de uso representan un elemento fundamental en el análisis y diseño de sistemas, ya que describen detalladamente cómo los distintos usuarios interactúan con la aplicación web de gestión de prácticas formativas. Permiten identificar y estructurar las funcionalidades del sistema de acuerdo con las necesidades específicas de cada perfil de usuario: alumno, tutor de empresa, tutor del centro y administrador. Estos casos de uso sirven como base tanto para el diseño y desarrollo del sistema como para la posterior validación del proyecto, asegurando que las funcionalidades implementadas cumplan con los objetivos planteados inicialmente. Además, facilitan la comunicación entre los distintos miembros del equipo de desarrollo, los profesores o tutores del proyecto, y los futuros usuarios de la aplicación, reduciendo la posibilidad de malentendidos y errores durante el ciclo de vida del software.

Los casos de uso permiten no solo visualizar el flujo de operaciones que cada usuario puede realizar, sino también comprender el alcance del sistema, priorizar funcionalidades según su importancia y planificar adecuadamente las etapas de implementación y prueba. Asimismo, ayudan a garantizar que el sistema sea intuitivo y eficiente, ofreciendo una experiencia de usuario coherente y satisfactoria.

2.3.1. DIAGRAMA DE CASOS DE USO

Un diagrama de casos de uso es un tipo de diagrama propio del lenguaje UML que representa gráficamente la interacción entre los distintos actores (usuarios o sistemas externos) y las funcionalidades principales del sistema. Cada caso de uso se define como una acción o un objetivo que un actor desea alcanzar, lo que permite entender de manera rápida y clara qué puede hacer cada tipo de usuario dentro de la aplicación.

Este tipo de diagrama es particularmente útil porque ayuda a definir el alcance y los límites del sistema, a clarificar los requisitos funcionales y a reducir la posibilidad de malentendidos entre desarrolladores, clientes, profesores y cualquier otra parte interesada. Además, proporciona una visión global de las operaciones disponibles y de cómo se interrelacionan, facilitando la identificación de dependencias y la planificación del desarrollo.

En la aplicación web de gestión de prácticas, el diagrama de casos de uso incluiría acciones como iniciar sesión, registrar horas de prácticas, validar registros, subir documentos, generar informes, evaluar alumnos, gestionar usuarios y asignar prácticas. Cada acción se asocia con uno o varios actores, permitiendo visualizar el funcionamiento general del sistema de forma clara, ordenada y fácilmente comprensible.

2.3.2. DESCRIPCIÓN DE LOS ACTORES

Los actores representan los distintos perfiles de usuarios que interactúan con el sistema. Cada actor tiene responsabilidades y permisos específicos que determinan qué acciones puede realizar:

- **Alumno:** Es el usuario que realiza las prácticas formativas. Puede registrar las horas de trabajo, subir documentos relacionados con la práctica, visualizar evaluaciones realizadas por tutores y consultar información sobre las prácticas asignadas. Su interacción con el sistema es constante y orientada al seguimiento de su progreso.
- **Tutor empresa:** Es el responsable de supervisar y validar las prácticas dentro de la empresa. Evalúa el desempeño del alumno, valida las horas registradas, sube informes de la empresa y puede realizar comentarios o sugerencias sobre la evolución de los estudiantes. Su rol es esencial para garantizar la correcta ejecución de las prácticas.
- **Tutor centro:** Es el representante del centro educativo y supervisa que las prácticas se desarrollen de acuerdo con los objetivos académicos. Valida evaluaciones, supervisa informes y genera estadísticas o informes sobre la evolución de los alumnos. También actúa como mediador entre la empresa y los alumnos si surgen incidencias.
- **Administrador:** Gestiona los usuarios del sistema, las empresas, las asignaciones de prácticas y configura las rúbricas de evaluación. Su interacción está centrada en la administración y mantenimiento del sistema, asegurando que los datos y permisos estén correctamente gestionados.

2.3.3. DESCRIPCIÓN DE LOS CASOS DE USO

Un caso de uso describe de manera detallada cómo un actor interactúa con el sistema para lograr un objetivo concreto. Incluye las condiciones previas necesarias, el flujo principal de acciones, las posibles excepciones o flujos alternativos y el resultado esperado. Este nivel de detalle permite visualizar claramente las funcionalidades desde la perspectiva del usuario, garantizando que el diseño y desarrollo del sistema respondan a necesidades reales y concretas.

Tabla 1 - Caso de uso (Iniciar sesión)

CU01	**Iniciar sesión**
Actor principal:	Alumno, Tutor Empresa, Tutor Centro, Administrador
Objetivo:	Permitir que el usuario acceda al sistema con su rol correspondiente.
Precondición:	El usuario debe estar registrado en el sistema.
Flujo principal:	1. El usuario ingresa su nombre de usuario y contraseña. 2. El sistema valida las credenciales. 3. El sistema redirige al panel correspondiente según el rol.
Flujo alternativo / Excepciones:	- Usuario o contraseña incorrectos → Mostrar mensaje de error. - Usuario inactivo → Denegar acceso y notificar al administrador.
Postcondición:	El usuario accede al panel de su rol si las credenciales son válidas.

Tabla 2 - Caso de uso (Registrar horas)

CU02	**Registrar horas de prácticas**
Actor principal:	Alumno
Objetivo:	Permitir que el alumno registre las horas realizadas durante las prácticas.
Precondición:	El alumno debe tener una práctica asignada.
Flujo principal:	1. El alumno accede a su panel. 2. Selecciona “Registrar horas”. 3. Introduce fecha, número de horas y descripción. 4. Guarda el registro. 5. El tutor de empresa podrá validarlo.
Flujo alternativo / Excepciones:	- Campos incompletos → Mostrar aviso. - Error al guardar → Mostrar mensaje de fallo. - No hay práctica asignada → Bloquear acción.
Postcondición:	El registro de horas queda guardado como “pendiente de validación”.

Tabla 3 - Caso de uso (Validar horas)

CU03	**Validar horas de prácticas**
Actor principal:	Tutor Empresa
Objetivo:	Validar las horas registradas por los alumnos.
Precondición:	El alumno debe haber registrado horas.
Flujo principal:	1. El tutor accede a su panel. 2. Consulta las horas registradas. 3. Selecciona las horas a validar. 4. Aprueba o rechaza el registro. 5. El sistema notifica al alumno.
Flujo alternativo / Excepciones:	- No hay registros disponibles → Mostrar mensaje. - Error al actualizar estado → Mostrar error. - Tutor sin permiso → Denegar acceso.
Postcondición:	Las horas quedan marcadas como “validadas” o “rechazadas”.

Tabla 4 - Caso de uso (Subir y gestionar documentos)

CU04	**Subir y gestionar documentos**
Actor principal:	Alumno, Tutor Empresa, Tutor Centro
Objetivo:	Permitir la subida, consulta y descarga de documentos asociados a la práctica.
Precondición:	El usuario debe estar autenticado y tener una práctica asignada.
Flujo principal:	1. El usuario selecciona la opción “Documentos”. 2. Sube un archivo (PDF, DOCX, etc.). 3. El sistema lo guarda y muestra la lista actualizada.
Flujo alternativo / Excepciones:	- Formato de archivo no permitido → Mostrar advertencia. - Error de red o servidor → Mostrar mensaje de error. - Documento duplicado → Mostrar aviso.
Postcondición:	El documento queda guardado y disponible para su consulta.

Tabla 5 - Caso de uso (Evaluar alumno)

CU05	**Evaluar alumno**
Actor principal:	Tutor Empresa, Tutor Centro
Objetivo:	Evaluar el desempeño del alumno durante las prácticas.
Precondición:	El alumno debe tener horas validadas y práctica activa.
Flujo principal:	1. El tutor accede al formulario de evaluación. 2. Introduce notas o comentarios. 3. Guarda la evaluación. 4. El sistema notifica al alumno y almacena los datos.
Flujo alternativo / Excepciones:	- Campos vacíos → Mostrar aviso. - Error de conexión → Mostrar mensaje. - Tutor no asignado → Bloquear acción.
Postcondición:	La evaluación queda registrada y visible para el alumno.

Tabla 6 - Caso de uso (Generar informes)

CU06	**Generar informes**
Actor principal:	Tutor Centro, Administrador
Objetivo:	Generar informes sobre las prácticas, alumnos o empresas.
Precondición:	Deben existir registros de prácticas y evaluaciones.
Flujo principal:	1. El usuario selecciona los filtros (alumno, empresa, fechas). 2. El sistema genera el informe en PDF o Excel. 3. El usuario descarga o visualiza el informe.
Flujo alternativo / Excepciones:	- Sin datos disponibles → Mostrar aviso. - Error al generar archivo → Mostrar mensaje de error.
Postcondición:	Informe generado correctamente.

Tabla 7 - Caso de uso (Gestionar)

CU07	**Gestionar usuarios, empresas y convenios**
Actor principal:	Administrador
Objetivo:	Administrar usuarios, empresas y convenios dentro del sistema.
Precondición:	Usuario autenticado como administrador.
Flujo principal:	1. Accede al módulo de administración. 2. Crea, edita o elimina registros. 3. Guarda cambios. 4. El sistema actualiza la base de datos.
Flujo alternativo / Excepciones:	- Datos incompletos → Mostrar error. - Usuario sin permisos → Acceso denegado. - Error de conexión a base de datos → Mostrar mensaje.
Postcondición:	Datos actualizados correctamente en el sistema.

Tabla 8 - Caso de uso (Editar perfil)

CU08	**Editar perfil de usuario**
Actor principal:	Alumno, Tutor Empresa, Tutor Centro, Administrador
Objetivo:	Permitir al usuario actualizar sus datos personales y de contacto.
Precondición:	El usuario debe estar autenticado en el sistema.
Flujo principal:	1. El usuario accede a su perfil desde el panel de rol. 2. Modifica la información deseada (nombre, email, contraseña, etc.). 3. Guarda los cambios.
Flujo alternativo / Excepciones:	- Campos incompletos → Mostrar advertencia. - Error al guardar → Mostrar mensaje de fallo
Postcondición:	Los datos del usuario quedan actualizados correctamente

Tabla 9 - Caso de uso (Asignar prácticas)

CU09	**Asignar prácticas a alumnos**
Actor principal:	Administrador.
Objetivo:	Asignar una práctica específica a uno o varios alumnos.
Precondición:	El alumno debe estar registrado y la práctica disponible.
Flujo principal:	1. El administrador accede al módulo de asignación de prácticas. 2. Selecciona el alumno y la práctica correspondiente. 3. Confirma la asignación.
Flujo alternativo / Excepciones:	- Práctica ya asignada → Mostrar mensaje de advertencia. - Error de conexión a base de datos → Mostrar mensaje de fallo.
Postcondición:	El alumno queda vinculado a la práctica y puede empezar a registrar horas.

2.4. MODELOS DE DATOS

El modelo de datos constituye una parte esencial en el diseño de cualquier sistema, ya que define la manera en que la información se organiza, almacena y relaciona dentro de la base de datos de la aplicación. Un modelo de datos bien estructurado permite que las operaciones de creación, lectura, actualización y eliminación (CRUD) se realicen de manera eficiente y coherente, evitando inconsistencias, redundancias o pérdida de información. Además, un buen diseño de datos facilita la escalabilidad del sistema, la generación de informes y la integración con otros módulos o aplicaciones.

En el contexto de la aplicación web de gestión de prácticas formativas, las entidades principales reflejan los elementos fundamentales del sistema, como alumnos, tutores, prácticas, empresas, documentos y evaluaciones. Cada entidad posee atributos que representan información relevante, y se establecen relaciones entre ellas para reflejar la realidad del proceso de gestión de prácticas.

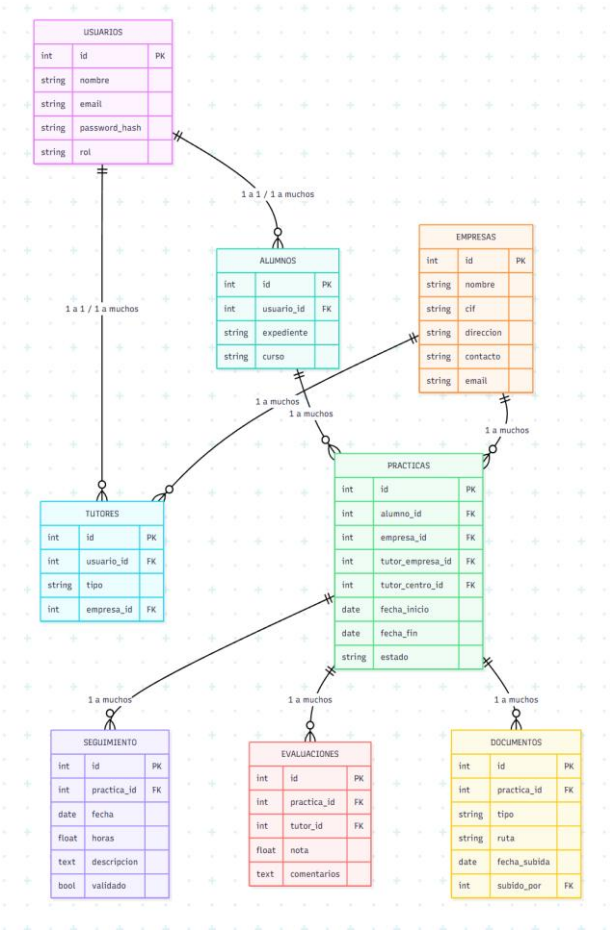
2.4.1. DIAGRAMA E/R

El **diagrama Entidad/Relación (E/R)** representa gráficamente cómo se relacionan las distintas entidades del sistema. Este diagrama es crucial para identificar la estructura de la base de datos y las dependencias entre los distintos elementos del sistema.

Las relaciones principales en esta aplicación son:

- **Alumno – Practica (uno a muchos):**
  Un alumno puede estar asignado a varias prácticas a lo largo de su formación, pero cada práctica está asociada únicamente a un alumno en particular. Esta relación permite registrar el historial completo de las prácticas de cada estudiante y su progreso en cada una.
- **Practica – Empresa (muchos a uno):**
  Varias prácticas pueden estar asociadas a una misma empresa, ya que una organización puede acoger a distintos alumnos en periodos diferentes. Esta relación facilita la gestión de convenios, asignaciones y reportes por empresa.
- **Usuario – Documento (uno a muchos):**
  Cada usuario (alumno o tutor) puede subir múltiples documentos relacionados con las prácticas, como informes, certificados, rúbricas de evaluación o evidencias de actividades. La relación garantiza un control adecuado de los archivos asociados a cada usuario.
- **Alumno – Evaluacion (uno a muchos):**
  Un alumno puede recibir múltiples evaluaciones a lo largo de diferentes prácticas. Cada evaluación queda registrada y vinculada al alumno correspondiente, permitiendo un seguimiento del desempeño individual a lo largo del tiempo.
- **Tutor – Evaluacion (uno a muchos):**
  Cada tutor puede evaluar a varios alumnos, y cada evaluación queda asociada al tutor que la realizó, lo que permite mantener un registro claro de responsabilidades y retroalimentación profesional.

Este diagrama proporciona una representación clara de cómo se conectan los distintos elementos del sistema, facilitando la comprensión de la estructura global de la base de datos y la detección de posibles redundancias o errores de diseño.



2.4.2. DIAGRAMA DE CLASES

El **diagrama de clases UML** representa las clases del sistema, sus atributos, métodos y las relaciones entre ellas. Es una herramienta de gran utilidad durante la fase de diseño, ya que permite:

- Visualizar la arquitectura del sistema de manera clara y comprensible.
- Organizar el diseño de las funcionalidades de manera modular.
- Facilitar la comunicación entre desarrolladores y otros miembros del proyecto.
- Detectar inconsistencias o errores de diseño antes de la implementación.

En esta aplicación, las principales clases incluyen:

- **Usuario:** clase base que contiene atributos comunes como nombre, correo electrónico, contraseña y rol.
    - **Subclases:**
        - **Alumno:** atributos específicos como número de matrícula, lista de prácticas y registros de horas.
        - **TutorEmpresa:** atributos relacionados con la empresa y las evaluaciones de los alumnos.
        - **TutorCentro:** atributos relacionados con la supervisión y generación de informes académicos.
        - **Administrador:** atributos de gestión y permisos del sistema.
- **Empresa:** contiene información sobre la organización, como nombre, dirección, sector y lista de prácticas ofrecidas.
- **Practica:** atributos como nombre de la práctica, descripción, fechas de inicio y fin, horas asignadas y alumno asignado.
- **Documento:** atributos de gestión de archivos como tipo, fecha de subida, usuario propietario y estado de validación.

Las relaciones reflejan la lógica de negocio del sistema: asignación de prácticas a alumnos, gestión de documentos por usuarios, evaluación de alumnos por tutores y vinculación de prácticas con empresas. Este modelo ofrece una visión completa de la estructura del software y permite planificar la implementación de manera eficiente y ordenada.

2.5. INTERFACES GRÁFICAS DE USUARIO

La **interfaz gráfica de usuario (GUI)** constituye el punto de contacto principal entre los usuarios y el sistema. Una GUI bien diseñada no solo facilita la navegación y el uso del sistema, sino que también contribuye a la satisfacción del usuario, reduce la probabilidad de errores y mejora la eficiencia en la realización de tareas.

En la aplicación de gestión de prácticas:

- La **página principal** actúa como panel de control, mostrando de manera clara las opciones disponibles según el rol del usuario, con accesos rápidos a funciones clave como registro de horas, validación de documentos y generación de informes.
- La **pantalla de login y registro** permite a los usuarios autenticarse de manera segura, recuperando contraseñas olvidadas y configurando perfiles iniciales.
- Los **perfiles de usuario** muestran información personal, documentos subidos, prácticas asignadas y evaluaciones, con opciones para actualizar datos y configurar notificaciones.

La interfaz debe ser intuitiva y accesible, con un diseño visual coherente que guíe al usuario en cada paso, minimizando la curva de aprendizaje y evitando errores comunes. Además, es recomendable que la GUI sea adaptable a distintos dispositivos (responsive design), para permitir el acceso desde ordenadores, tablets y teléfonos móviles, garantizando así la flexibilidad y disponibilidad del sistema en cualquier contexto.

Ilustración 2 - MockUp página principal

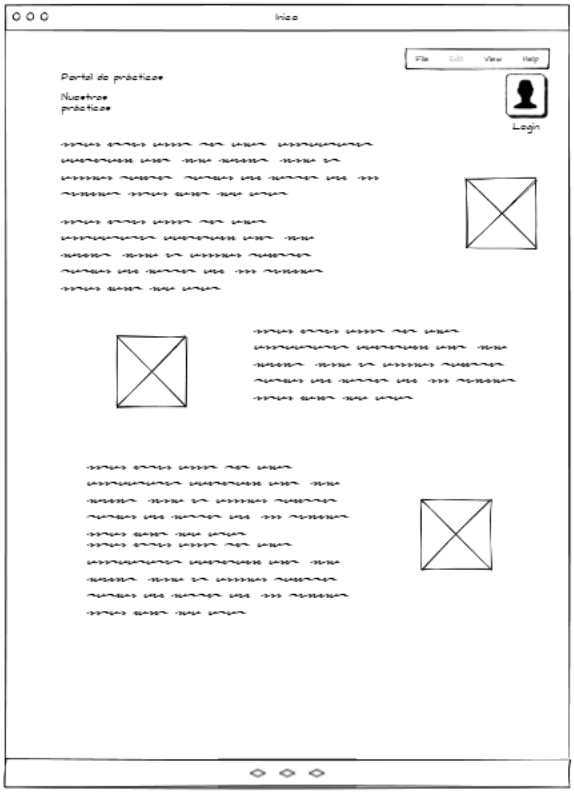


Ilustración 3 - MockUp login y registrarse

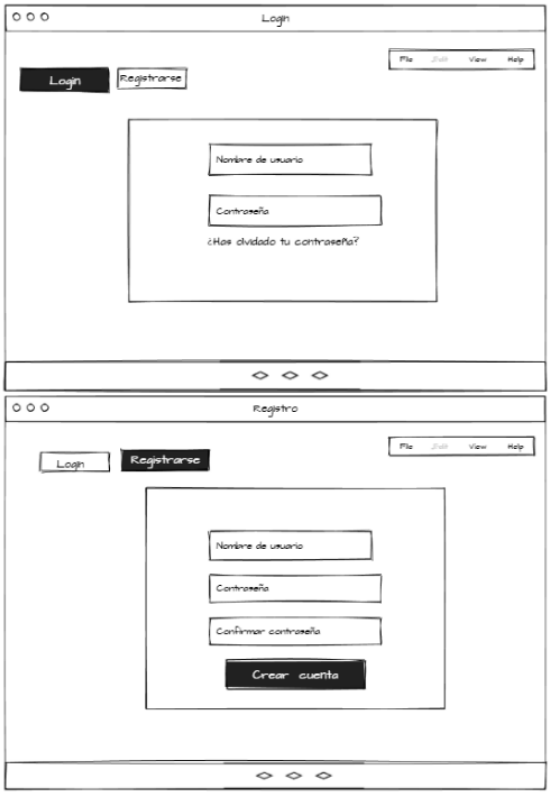
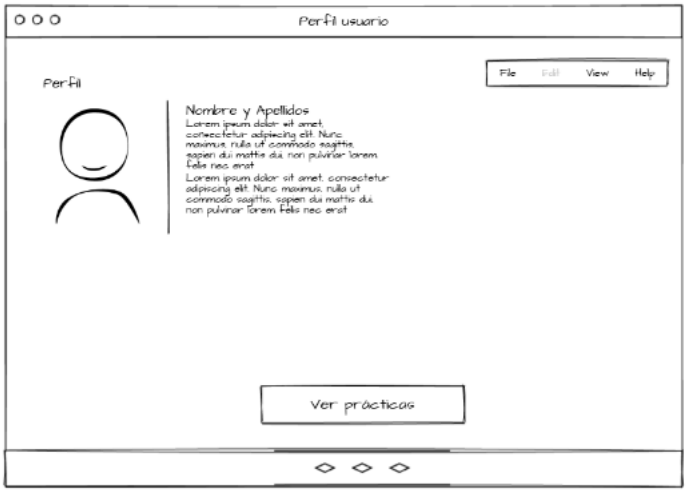


Ilustración 4 - MockUp perfil



2.5.1. MAPA DE NAVEGABILIDAD

El **mapa de navegabilidad** es una representación visual que muestra cómo se conectan las diferentes pantallas de la aplicación web y los posibles recorridos que puede seguir un usuario según su rol. Este mapa es esencial porque permite planificar la estructura de la interfaz de manera lógica, asegurando que los usuarios puedan realizar sus tareas sin confusión ni pasos innecesarios.

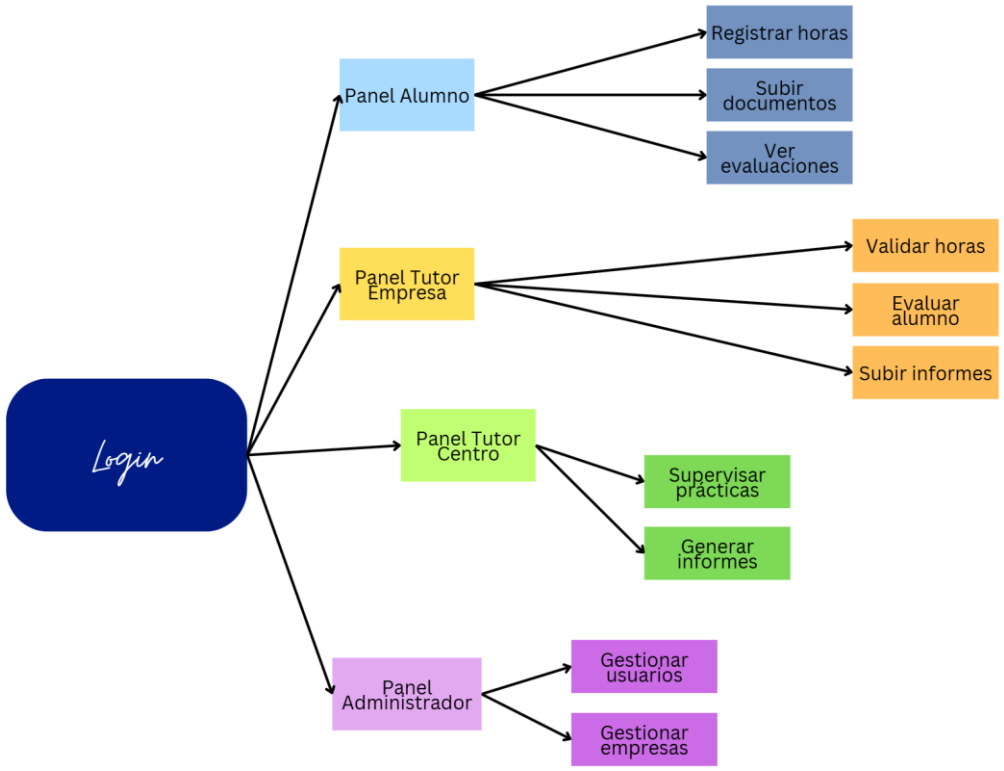
En la práctica, un mapa de navegabilidad incluye:

- **Pantallas principales**: panel de control, perfil de usuario, lista de prácticas, subida de documentos, evaluaciones y generación de informes.
- **Accesos rápidos**: botones o enlaces que permiten pasar de una sección a otra sin necesidad de regresar al menú principal.
- **Flujos condicionales**: rutas que dependen del rol del usuario (alumno, tutor o administrador) o del estado de los datos (por ejemplo, horas pendientes de validación).

El mapa de navegabilidad ayuda a detectar problemas antes de implementar la interfaz, como ciclos de navegación innecesarios, pantallas inaccesibles o funciones duplicadas. Además, sirve como guía para los desarrolladores y diseñadores de la GUI, asegurando coherencia en toda la aplicación.

Ilustración 6 - Mapa de navegabilidad
(Un ejemplo incluiría un diagrama donde se vea la página de inicio conectando con los módulos de registro de horas, documentos, evaluaciones, informes y administración, mostrando las rutas posibles según el rol del usuario.)

Ilustración 5 - Mapa de navegabilidad



3. PLAN DE PRUEBAS

El plan de pruebas tiene como objetivo garantizar que la aplicación funcione correctamente en un entorno real, verificando que cada funcionalidad cumpla con los requisitos establecidos y ofrezca una experiencia de usuario óptima. A diferencia de enfoques basados en herramientas de testing automatizado, las pruebas de este proyecto se han llevado a cabo directamente sobre la aplicación desplegada, navegando por la interfaz de usuario e interactuando con el sistema de la misma forma en que lo haría un usuario final.

Este enfoque permite detectar errores reales de funcionamiento, problemas de navegación, fallos en la interfaz y comportamientos inesperados en condiciones de uso auténticas.

3.1. Pruebas funcionales

Las pruebas funcionales verifican que cada funcionalidad del sistema cumpla su propósito según lo definido en los casos de uso. Se han realizado accediendo directamente a la aplicación desplegada y ejecutando cada acción de forma manual.

Autenticación y control de acceso por roles. Se ha comprobado que cada tipo de usuario (alumno, tutor de empresa, tutor de centro y administrador) puede iniciar sesión correctamente con sus credenciales y es redirigido al panel correspondiente a su rol. Asimismo, se ha verificado que un usuario no puede acceder a rutas que no le corresponden, recibiendo en su lugar una respuesta de acceso denegado.

Registro y validación de horas. Se ha probado que los alumnos pueden registrar horas desde su panel, introduciendo fecha, duración y descripción de las tareas. Posteriormente se ha comprobado que el tutor de empresa visualiza correctamente esos registros y puede aprobarlos o rechazarlos, reflejándose el cambio de estado de forma inmediata en la vista del alumno.

Subida y consulta de documentos. Se ha verificado que los usuarios pueden subir archivos desde la interfaz, que estos quedan almacenados correctamente y que pueden consultarse y descargarse sin errores. También se ha comprobado el comportamiento ante formatos no permitidos.

Evaluaciones. Se ha probado que los tutores pueden acceder al formulario de evaluación, introducir calificaciones y comentarios, y guardarlos correctamente. Se ha verificado que la evaluación queda visible en el perfil del alumno correspondiente.

Generación de informes. Se ha comprobado que los informes se generan correctamente aplicando distintos filtros, y que el archivo resultante puede descargarse sin errores.

3.2. Pruebas de integración

Estas pruebas verifican que los distintos módulos de la aplicación funcionan de manera conjunta y coherente en el entorno desplegado.

Se ha comprobado que los registros de horas se vinculan correctamente al alumno y a la práctica asignada, que las evaluaciones completadas aparecen reflejadas en los informes generados, y que los documentos subidos quedan asociados al usuario y práctica correspondientes. También se ha verificado la coherencia de los datos entre las distintas vistas de la aplicación tras realizar operaciones de creación, edición y eliminación.

3.3. Pruebas de usabilidad

Las pruebas de usabilidad se han realizado navegando por la aplicación desplegada con el objetivo de evaluar la experiencia de usuario en condiciones reales.

Se ha comprobado que la navegación entre pantallas es fluida e intuitiva, que los formularios están correctamente etiquetados y son fáciles de completar, y que los mensajes de error son comprensibles y orientan al usuario sobre cómo resolver el problema. También se ha verificado que el diseño se adapta correctamente a distintos tamaños de pantalla, incluyendo dispositivos móviles y tablets.

3.4. Pruebas de seguridad

Se han realizado pruebas de seguridad básicas directamente desde el navegador para verificar que el sistema protege correctamente el acceso a la información.

Se ha comprobado que las rutas protegidas no son accesibles sin autenticación previa, redirigiendo al usuario a la pantalla de login. Se ha verificado que un usuario autenticado con un rol determinado no puede acceder a rutas reservadas para otros roles. También se ha comprobado que los formularios incluyen protección CSRF, evitando envíos no autorizados, y que las contraseñas se almacenan cifradas en la base de datos, no siendo visibles en ningún momento desde la interfaz ni en los registros del sistema.

4. TECNOLOGÍAS EMPLEADAS

El desarrollo de la aplicación se ha llevado a cabo utilizando un conjunto de tecnologías modernas, ampliamente adoptadas en el sector profesional y seleccionadas por su robustez, seguridad y facilidad de integración entre sí.

4.1. Backend

Java 17 es el lenguaje de programación principal del proyecto. Se trata de un lenguaje orientado a objetos, fuertemente tipado y con una amplia comunidad de soporte, especialmente indicado para aplicaciones empresariales que requieren estabilidad y escalabilidad a largo plazo.

Spring Boot 3 es el framework principal sobre el que se construye la aplicación. Permite desarrollar aplicaciones web con una configuración mínima gracias a su principio de convención sobre configuración, integrando de forma transparente todos los módulos necesarios para el proyecto.

Spring Security es el módulo encargado de gestionar la autenticación y autorización del sistema. Permite configurar el acceso a las distintas rutas de la aplicación en función del rol del usuario, proteger los formularios mediante tokens CSRF y gestionar el flujo completo del proceso de login, incluyendo el manejo de errores y redirecciones personalizadas según el perfil.

Spring Data JPA con Hibernate proporciona una capa de abstracción sobre la base de datos relacional, permitiendo trabajar con entidades Java directamente sin necesidad de escribir consultas SQL manualmente en la mayoría de los casos. Hibernate actúa como implementación del estándar JPA, gestionando el mapeo objeto-relacional de forma eficiente.

Maven es la herramienta de construcción y gestión de dependencias del proyecto. Permite definir todas las librerías necesarias en el archivo `pom.xml` y automatiza la compilación, el empaquetado y la ejecución de pruebas.

4.2. Frontend

HTML5 define la estructura semántica de todas las vistas de la aplicación, organizando los elementos de forma clara y accesible.

CSS3 se encarga del diseño visual, incluyendo tipografía, colores, disposición de elementos y adaptación a distintos tamaños de pantalla mediante media queries.

JavaScript aporta interactividad a la interfaz, permitiendo validaciones en el lado del cliente, comportamientos dinámicos y mejoras en la experiencia de usuario sin necesidad de recargar la página.

Thymeleaf es el motor de plantillas utilizado para integrar la lógica del backend directamente en las vistas HTML. Permite renderizar datos dinámicos desde el servidor, gestionar fragmentos reutilizables, mostrar mensajes condicionales y trabajar con variables del modelo de Spring MVC de forma natural y segura. Es fundamental para funcionalidades como la protección CSRF en formularios o la visualización de errores de autenticación.

4.3. Base de datos

MySQL es el sistema gestor de bases de datos relacional utilizado para almacenar toda la información del sistema: usuarios, roles, prácticas, empresas, registros de horas, evaluaciones y documentos. Se ha elegido por su amplia adopción, rendimiento probado en aplicaciones web y buena integración con Spring Data JPA.

4.4. Herramientas de desarrollo

Eclipse es el entorno de desarrollo integrado (IDE) principal utilizado durante el proyecto. Ofrece soporte avanzado para Java y Spring, con funcionalidades como autocompletado inteligente, depuración integrada, gestión de dependencias Maven y herramientas de refactorización que agilizan el desarrollo.

MySQL Workbench ha servido como herramienta visual para el diseño, gestión y consulta de la base de datos durante el desarrollo, facilitando la creación de tablas, la importación de scripts SQL y la verificación de datos en tiempo real.

Git y GitHub se han utilizado para el control de versiones del proyecto, permitiendo registrar el historial de cambios, trabajar con ramas para el desarrollo de nuevas funcionalidades y mantener una copia de seguridad del código en remoto.

Todo el desarrollo del proyecto ha sido versionado mediante Git y almacenado en un repositorio remoto en GitHub. Esto ha permitido llevar un control detallado de los cambios realizados a lo largo del desarrollo, facilitando la gestión de versiones, la recuperación de estados anteriores del proyecto y el seguimiento de la evolución del sistema. Además, el uso de GitHub ha servido como copia de seguridad del código y como herramienta de organización del trabajo.

ArDuNqUeN/Proyecto_Diaz_Perez_Aidan

5. PLANIFICACIÓN Y PRESUPUESTO

El desarrollo del proyecto se ha realizado siguiendo una planificación estructurada por fases, diseñada para garantizar la correcta organización del trabajo y la consecución de los objetivos en tiempo y forma. Cada fase tiene tareas específicas y entregables definidos, asegurando que el proyecto avance de manera coherente y controlada.

Fases del proyecto

Fase 1 — Análisis de requisitos. Se identificaron las necesidades de los distintos tipos de usuarios del sistema y se definieron los objetivos generales y específicos del proyecto. Se documentaron los casos de uso, los actores implicados y las funcionalidades esenciales, sentando las bases para el diseño y desarrollo posteriores.

Fase 2 — Diseño del sistema. Se llevó a cabo el modelado de datos mediante diagramas Entidad/Relación y diagramas de clases UML. También se diseñó la interfaz de usuario a través de mockups y se definió el mapa de navegabilidad, estableciendo el flujo de interacción entre las distintas pantallas de la aplicación.

Fase 3 — Desarrollo backend. Se implementó la lógica de negocio del sistema, incluyendo servicios, controladores, configuración de seguridad con Spring Security y persistencia de datos mediante Spring Data JPA con Hibernate. En esta fase se configuraron los roles de usuario, las rutas protegidas, el sistema de autenticación y las operaciones CRUD sobre las entidades principales.

Fase 4 — Desarrollo frontend. Se crearon las vistas de la aplicación utilizando HTML5, CSS3 y JavaScript, integrando Thymeleaf como motor de plantillas para conectar la interfaz con los datos del backend. Se aplicó un diseño responsive y coherente con la guía de estilos definida para el proyecto.

Fase 5 — Integración. Se realizó la conexión completa entre el frontend y el backend, verificando que todos los módulos funcionaran de manera conjunta. Se comprobó la correcta comunicación entre las vistas, los controladores y la base de datos en los distintos flujos de la aplicación.

Fase 6 — Pruebas. Se llevaron a cabo pruebas funcionales, de integración, de usabilidad y de seguridad directamente sobre la aplicación desplegada, simulando el uso real del sistema por parte de los distintos perfiles de usuario.

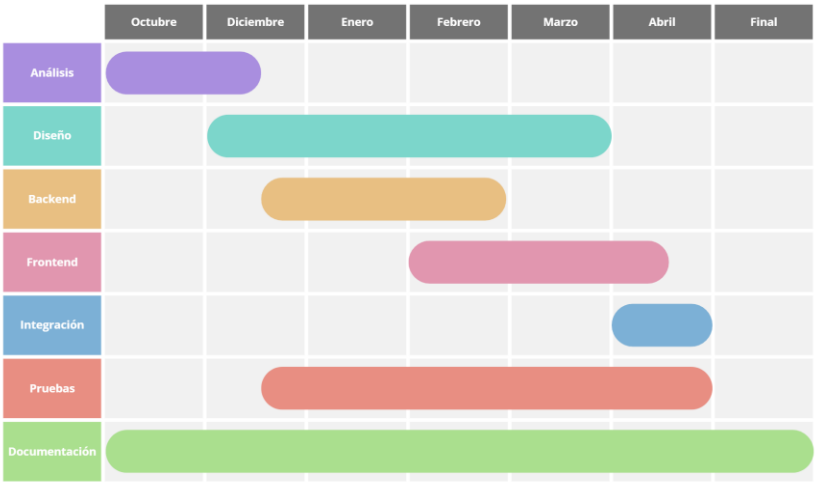
Fase 7 — Documentación. Se elaboró la documentación técnica del proyecto, incluyendo la memoria, el manual de usuario, los diagramas del sistema y la guía de estilos, con el objetivo de facilitar el mantenimiento y futuras ampliaciones de la aplicación.

5.1. DIAGRAMA DE GANTT

El **diagrama de Gantt** representa gráficamente la planificación temporal del proyecto, mostrando la duración de cada fase, los hitos y la superposición de tareas. Esta visualización permite:

- Identificar dependencias entre tareas.
- Detectar posibles retrasos y ajustar los plazos.
- Planificar recursos humanos y materiales.

Ilustración 6 - Diagrama de Gantt



Las fases principales del proyecto y su duración aproximada han sido las siguientes:

La fase de análisis y diseño se desarrolló durante las primeras 5 meses, abarcando la definición de requisitos, el diseño del modelo de datos, la arquitectura del sistema y los mockups de las interfaces. La fase de desarrollo del backend ocupó los meses desde diciembre hasta febrero, implementando los módulos de autenticación, gestión de usuarios, prácticas, registro de horas y documentos. El desarrollo del frontend se realizó en paralelo con el backend a partir del mes de febrero, integrando las plantillas Thymeleaf con los controladores Spring. La fase de pruebas se desarrollará en el mes de abril, incluyendo pruebas unitarias, de integración y funcionales. La documentación fue un proceso que se empezó desde el inicio del proyecto hasta la finalizació del mismo.

5.2. ESTIMACIONES DE COSTES

Aunque el proyecto tiene carácter académico, es posible realizar una estimación del coste que tendría en un contexto profesional real, así como definir una estrategia de comercialización viable a medio plazo.

Costes de desarrollo

Tabla 10 - Tabla de costes

Concepto	Detalle	Coste estimado
Desarrollo	Horas utilizadas el desarrollo de la aplicación	3.000 €
Infraestructura	Hosting, dominio y servidor	100 €
Herramientas de software	Software libre y licencias gratuitas	0 €
Total		**3.100 €**

Estrategia de comercialización

La estrategia de negocio planteada para PracticApps se basa en un modelo de crecimiento progresivo en dos fases diferenciadas.

En una primera fase, la aplicación se ofrecería a instituciones de educación superior privadas a un precio reducido, con el objetivo de captar un volumen inicial de clientes y, sobre todo, de alimentar el sistema con datos reales: empresas colaboradoras, convenios, alumnos y prácticas. Esta fase permite validar el producto en un entorno real, detectar mejoras necesarias e incrementar el valor del sistema de forma orgánica a medida que crece la base de datos de empresas registradas.

Una vez consolidada esta base de datos y ampliadas las funcionalidades de la plataforma, se daría paso a una segunda fase orientada a instituciones de educación pública, donde el sistema ya contaría con un ecosistema de empresas colaboradoras establecido, lo que representa un valor añadido diferencial difícil de replicar desde cero. En este contexto, el modelo de negocio contempla un precio de licencia de **5.000 € por institución**, lo que permitiría recuperar la inversión inicial y generar rentabilidad a partir de un número relativamente bajo de clientes.

Este enfoque, conocido como modelo freemium de entrada o estrategia de penetración de mercado, es habitual en soluciones SaaS dirigidas al sector educativo, donde la confianza y los datos acumulados son factores clave en la decisión de compra.

6. FUTURAS MEJORAS Y AMPLIACIONES

Como ocurre en la mayoría de las aplicaciones web actuales, PracticApps está diseñada para evolucionar a lo largo del tiempo. Las aplicaciones no son productos estáticos, sino que se actualizan constantemente para adaptarse a nuevas necesidades, mejorar su rendimiento y corregir posibles errores. En el contexto del modelo de negocio planteado, estas mejoras cobran especial relevancia, ya que el valor comercial de la plataforma está directamente ligado a la amplitud y calidad de sus funcionalidades.

6.1. PROPUESTAS DE MEJORA Y AMPLIACIONES

Aplicación móvil nativa (Android/iOS):

Dado que alumnos y tutores de empresa interactúan con el sistema de forma frecuente y desde entornos variados, el desarrollo de una aplicación móvil nativa permitiría registrar horas, validar documentos y consultar evaluaciones desde cualquier lugar, mejorando significativamente la accesibilidad y la experiencia de usuario.

Sistema de mensajería en tiempo real:

Reemplazar el sistema de notificaciones actual por un sistema de mensajería en tiempo real basado en WebSockets (usando Spring WebSocket y STOMP) permitiría la comunicación instantánea entre alumnos y tutores, similar a un chat.

Firma digital de documentos:

Incorporar un sistema de firma digital permitiría validar convenios, informes y otros documentos oficiales de forma completamente digital, eliminando la necesidad de gestión en papel y dotando al sistema de mayor validez legal y trazabilidad.

Integración con plataformas educativas:

La integración con plataformas como Moodle o Google Classroom permitiría sincronizar automáticamente la información de los alumnos y las evaluaciones, evitando la duplicación de datos y simplificando la gestión para los centros que ya utilizan estas plataformas.

Sistema de mensajería interna:

Incorporar un módulo de comunicación directa entre usuarios dentro de la propia plataforma mejoraría la coordinación entre alumnos, tutores y administradores, centralizando toda la gestión de las prácticas en un único entorno.

Mejora del sistema de informes:

Ampliar las capacidades de generación de informes con opciones avanzadas de filtrado, visualización de datos mediante gráficas y estadísticas comparativas aportaría a los centros educativos una herramienta de análisis mucho más potente para la toma de decisiones.

.

Inteligencia artificial para evaluaciones:

Implementar un módulo de análisis de las actividades registradas por los alumnos usando procesamiento de lenguaje natural (NLP) podría ayudar a los tutores a identificar patrones, detectar alumnos en dificultades y generar sugerencias de evaluación automáticas.

Dashboard de analítica avanzada:

Desarrollar un panel de analítica con métricas agregadas sobre el rendimiento general del programa de prácticas, las empresas colaboradoras más activas, las tasas de validación de horas y la evolución de las evaluaciones a lo largo del tiempo.

Multiidioma:

Añadir soporte para múltiples idiomas usando el sistema de internacionalización de Spring Boot, para dar cabida a centros con alumnado internacional.

Integración con APIs externas:

El uso de servicios externos permitiría incorporar funcionalidades como el envío automático de correos electrónicos, el almacenamiento de documentos en la nube o la autenticación mediante proveedores de identidad institucionales, aumentando la robustez y flexibilidad del sistema

6.2. IMPACTO Y RIESGO DE LOS CAMBIOS PROPUESTOS

Impacto positivo

La incorporación de estas mejoras tendría un efecto directo en el valor percibido de la plataforma, tanto por parte de los usuarios finales como de las instituciones que la adopten. Entre los beneficios más destacados se encuentran la mayor eficiencia en la gestión de prácticas y la reducción de errores administrativos, la mejora en la comunicación y coordinación entre todos los actores del sistema, una mayor flexibilidad de acceso gracias a la versión móvil, y un incremento general en la satisfacción del usuario y en la productividad del centro educativo. En el contexto del modelo de negocio planteado, estas ampliaciones reforzarían además el argumento de venta frente a nuevas instituciones, justificando el precio de licencia establecido.

Riesgos potenciales

Todo proceso de ampliación conlleva riesgos que deben gestionarse de forma planificada. La incorporación de nuevas funcionalidades puede introducir vulnerabilidades de seguridad si no se implementan siguiendo buenas prácticas. Un crecimiento no optimizado podría derivar en una sobrecarga del sistema y una disminución del rendimiento. Asimismo, una mayor complejidad funcional implica una mayor carga de mantenimiento y la necesidad de actualizaciones y supervisión continuas para garantizar la calidad y estabilidad del sistema a lo largo del tiempo.

7. RELACIÓN CON LOS MÓDULOS DEL CICLO

El desarrollo de este proyecto ha requerido la aplicación de conocimientos y competencias adquiridos a lo largo de los diferentes módulos del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web. A continuación se detalla la relación entre cada módulo y su aplicación concreta en el proyecto:

Módulos de segundo curso

Desarrollo Web en Entorno Servidor (DWES):

Es el módulo central del proyecto. Todo el backend de la aplicación está construido sobre Spring Boot, aplicando los conceptos de arquitectura MVC, creación de controladores y servicios, gestión de sesiones, control de acceso por roles mediante Spring Security y persistencia de datos con JPA e Hibernate. La configuración del sistema de login, la protección de rutas y la lógica de negocio son el resultado directo de los conocimientos adquiridos en este módulo.

Desarrollo Web en Entorno Cliente (DWEC):

Los conocimientos de JavaScript, HTML5 y CSS3 han sido esenciales para el desarrollo de las plantillas Thymeleaf y la interactividad de la interfaz. Se han aplicado en el diseño y comportamiento de todas las vistas de la aplicación, incluyendo validaciones en el lado del cliente, interactividad en los formularios y una interfaz adaptada a distintos dispositivos mediante diseño responsive.

Despliegue de Aplicaciones Web:

Este módulo ha sido clave para llevar la aplicación de un entorno de desarrollo local a un entorno desplegado real, configurando el servidor, gestionando las dependencias y asegurando el correcto funcionamiento del sistema en producción. Las pruebas del proyecto se han realizado directamente sobre la aplicación desplegada, aplicando los conocimientos adquiridos en esta asignatura.

Diseño de Interfaces Web:

Los principios de usabilidad, accesibilidad y diseño visual trabajados en este módulo se reflejan en la guía de estilos de PracticApps, en la coherencia visual de las vistas y en la estructuración lógica de la navegación, buscando siempre una experiencia de usuario intuitiva y agradable.

Ciberseguridad:

Los conceptos de seguridad aplicada han guiado decisiones clave del proyecto, como el cifrado de contraseñas, la protección de formularios mediante tokens CSRF, la validación de accesos por rol y la protección de rutas frente a usuarios no autenticados. Este módulo ha aportado una visión crítica sobre los riesgos potenciales del sistema y las medidas necesarias para mitigarlos.

Itinerario para la Empleabilidad:

Los conocimientos de economía y emprendimiento adquiridos en este módulo han permitido desarrollar una visión estratégica sobre la viabilidad comercial del proyecto, plasmada en el modelo de negocio definido en la sección de planificación y presupuesto, donde se contempla una estrategia de penetración de mercado dirigida a instituciones educativas privadas y públicas.

Módulos de primer curso

Bases de Datos:

El diseño del modelo de datos, la creación del diagrama E/R, la normalización de las tablas y la escritura de consultas SQL han requerido una aplicación directa de los contenidos de este módulo. El uso de Hibernate como ORM no elimina la necesidad de entender el modelo relacional subyacente; al contrario, un buen diseño de la base de datos es fundamental para el rendimiento y la integridad de la aplicación.

Lenguajes de Marcas:

El uso correcto de HTML5 como lenguaje estructural de las vistas, junto con la integración de Thymeleaf para la generación dinámica de contenido, tiene su base en los fundamentos trabajados en este módulo.

Entornos de Desarrollo:

El uso de Eclipse como IDE principal, la gestión de dependencias con Maven, el control de versiones con Git y GitHub, y la aplicación de buenas prácticas en la organización del código son competencias desarrolladas en este módulo y aplicadas de forma constante a lo largo de todo el proyecto.

Programación:

Los fundamentos de la programación orientada a objetos en Java, incluyendo el uso de clases, herencia, encapsulamiento e interfaces, constituyen la base sobre la que se ha construido toda la lógica del backend de la aplicación.

Sistemas Informáticos:

Los conocimientos sobre sistemas operativos, redes y configuración de entornos han sido de utilidad durante el proceso de despliegue y configuración del servidor donde se aloja la aplicación.

8. CONCLUSIONES

El desarrollo de este proyecto ha supuesto un reto técnico y personal que ha permitido consolidar y ampliar los conocimientos adquiridos a lo largo del ciclo formativo. La creación de una aplicación web completa, desde el análisis de requisitos hasta el despliegue en producción, ha proporcionado una visión integral del ciclo de vida del desarrollo software que difícilmente se obtiene de forma aislada en los módulos individuales.

Desde el punto de vista técnico, el proyecto ha permitido profundizar en el uso del framework Spring Boot y su ecosistema, descubriendo las ventajas que ofrece en términos de productividad y mantenibilidad. La implementación de Spring Security ha sido especialmente enriquecedora, ya que ha obligado a comprender en profundidad los mecanismos de autenticación y autorización en aplicaciones web. El diseño del modelo de datos y su implementación con Hibernate ha reforzado la importancia de una buena arquitectura de datos como fundamento de cualquier aplicación robusta.

Desde una perspectiva metodológica, la adopción de un enfoque ágil adaptado ha resultado útil para gestionar los cambios propios de los proyectos de desarrollo. La organización en sprints cortos ha facilitado la detección y corrección temprana de desviaciones, evitando que los problemas se acumulen en fases finales.

El principal desafío encontrado durante el desarrollo ha sido la integración de Spring Security con el sistema de roles personalizado, que requirió un estudio en profundidad de la documentación y varios ciclos de prueba y error.

En cuanto a los objetivos planteados al inicio del proyecto, los objetivos principales se encuentran en proceso de desarrollo y presentan un grado de avance satisfactorio. La aplicación ya permite acceder a diferentes páginas en función del tipo de usuario. El sistema está desplegado y accesible desde cualquier navegador web en local.

Como reflexión final, este proyecto pone de manifiesto que el desarrollo de software es una disciplina que no solo requiere conocimientos técnicos sólidos, sino también capacidad de planificación, resolución de problemas, comunicación y aprendizaje continuo. Además, dado que las tecnologías están en constante evolución, resulta especialmente importante desarrollar la capacidad de aprender y adaptarse a nuevas herramientas y enfoques.

9. DESPLIEGUE DE LA APLICACIÓN

La aplicación PracticApps puede ejecutarse tanto en un servidor local como en un servidor remoto, dependiendo del entorno de uso. Esta flexibilidad permite utilizarla tanto para pruebas y demostraciones durante el desarrollo como para un despliegue real en producción en un centro educativo.

9.1. Estructura del proyecto y elección de Spring Boot

Una de las primeras decisiones técnicas del proyecto fue elegir Spring Boot como framework principal para el backend. Esta elección no fue arbitraria, sino que responde a una serie de ventajas concretas que han tenido un impacto directo en la velocidad de desarrollo, la organización del código y la facilidad de despliegue.

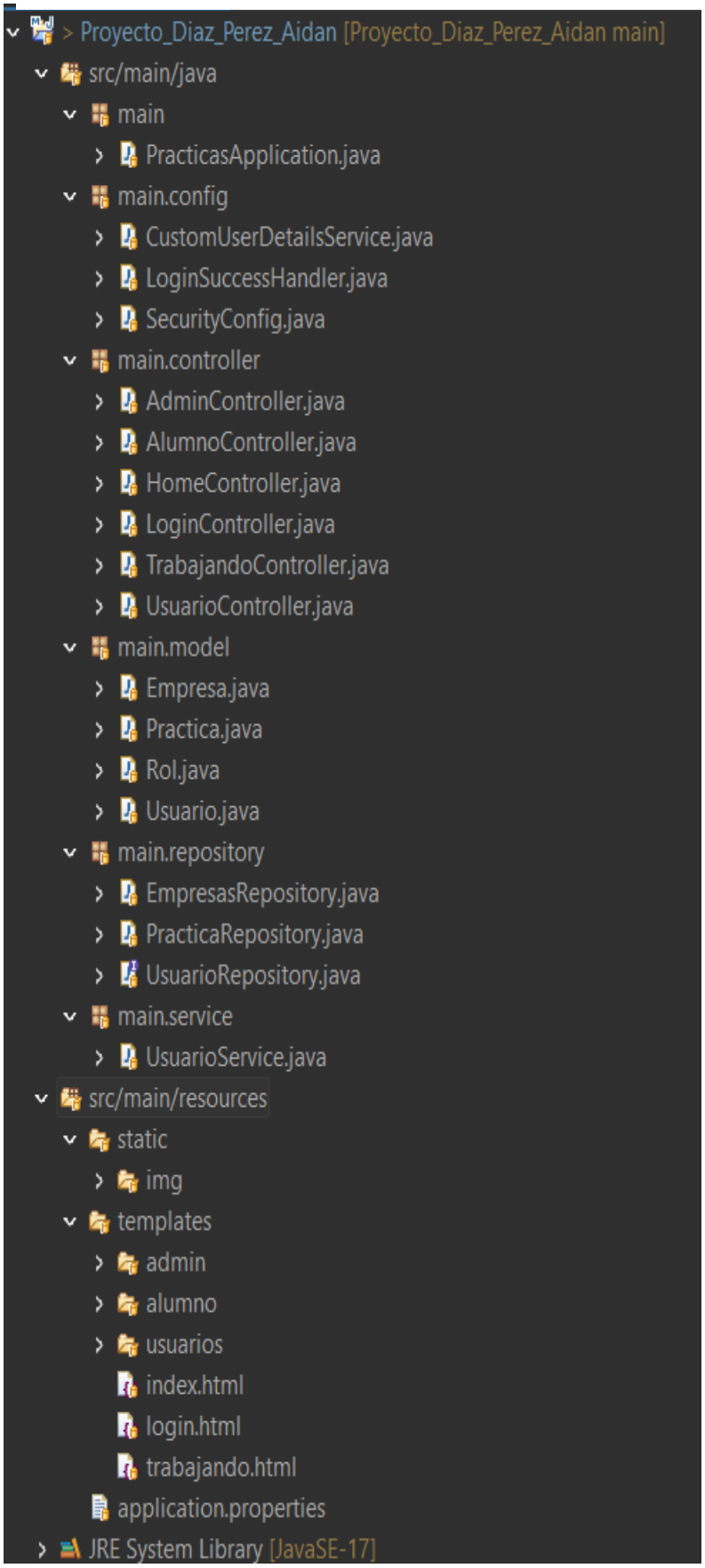
Spring Boot sigue el patrón de arquitectura MVC (Modelo-Vista-Controlador), lo que permite separar de forma clara las responsabilidades del sistema: los modelos representan las entidades de la base de datos, los controladores gestionan las peticiones del usuario y las vistas se encargan de presentar la información. Esta separación hace que el código sea más limpio, más fácil de mantener y más sencillo de ampliar en el futuro.

Entre las principales ventajas que ha aportado Spring Boot al proyecto destacan las siguientes. En primer lugar, la **autoconfiguración**: Spring Boot es capaz de configurar automáticamente la mayoría de los componentes del sistema en función de las dependencias declaradas en el archivo `pom.xml`, eliminando la necesidad de configuraciones manuales extensas que serían necesarias con otros frameworks. En segundo lugar, el **servidor embebido**: la aplicación incluye un servidor Tomcat integrado, lo que significa que no es necesario instalar ni configurar un servidor externo para ejecutarla, simplificando enormemente el proceso de despliegue. En tercer lugar, la **gestión automática de la base de datos**: gracias a Spring Data JPA e Hibernate, el sistema es capaz de generar automáticamente las tablas de la base de datos a partir de las clases Java definidas como entidades, sin necesidad de escribir scripts SQL manualmente. Esto reduce los errores de sincronización entre el modelo de datos y la base de datos real. Por último, la **integración nativa con Spring Security** permite configurar la autenticación, los roles y la protección de rutas de forma declarativa y centralizada, con un nivel de seguridad robusto y sin necesidad de implementar estos mecanismos desde cero.

La estructura del proyecto sigue las convenciones estándar de Spring Boot, organizando el código en paquetes por responsabilidad:

La estructura del proyecto sigue el estándar de cualquier aplicación Spring Boot, organizando el código en capas bien diferenciadas según su responsabilidad

Ilustración 7 -Estructura del proyecto



PracticasApplication.java es el punto de entrada de la aplicación. Contiene el método main, que es el que ejecuta Spring Boot y pone en marcha todo el sistema al arrancar.

config agrupa las clases de configuración de la aplicación. En este proyecto incluye la configuración de Spring Security, donde se definen las rutas protegidas, los roles de acceso, el comportamiento del formulario de login y el manejador de redirección tras autenticarse correctamente.

controller contiene los controladores de la aplicación. En Spring Boot, cada controlador es el responsable de escuchar las peticiones HTTP que llegan desde el navegador y devolver la vista correspondiente o los datos necesarios. Es el intermediario entre lo que el usuario solicita y lo que el sistema responde.

model define las entidades del sistema, es decir, las clases Java que representan los objetos del mundo real como usuarios, prácticas, empresas o evaluaciones. Cada entidad se mapea automáticamente a una tabla de la base de datos gracias a JPA e Hibernate, por lo que no es necesario crear las tablas manualmente.

service es la capa donde reside la lógica de negocio de la aplicación. Los servicios actúan como intermediarios entre los controladores y los repositorios, procesando los datos, aplicando las reglas del sistema y coordinando las operaciones necesarias antes de devolver una respuesta.

Dentro de **resources** se encuentran dos carpetas principales. La carpeta **static** almacena los recursos estáticos del frontend: hojas de estilo CSS, scripts JavaScript e imágenes. La carpeta **templates** contiene los archivos HTML de las vistas, que utilizan Thymeleaf para mostrar datos dinámicos enviados desde el backend.

Como se puede observar en la imagen, el proyecto se organiza en los siguientes paquetes principales dentro de `src/main/java`: el paquete `config` contiene las clases de configuración de seguridad y otros componentes transversales; el paquete `controller` agrupa los controladores que gestionan las peticiones HTTP; el paquete `model` contiene las entidades JPA que representan las tablas de la base de datos; el paquete `repository` define las interfaces de acceso a datos; y el paquete `service` implementa la lógica de negocio de la aplicación. Las vistas Thymeleaf se encuentran en `src/main/resources/templates`, y los recursos estáticos como hojas de estilo y scripts en `src/main/resources/static`.

Ilustración 8 - Propiedades del proyecto

```
application.properties
1  spring.datasource.url=jdbc:h2:mem:testdb
2  spring.datasource.driverClassName=org.h2.Driver
3  spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
4  spring.h2.console.enabled=true
5  spring.jpa.hibernate.ddl-auto=update
6  spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.r2dbc.R2dbcAutoConfiguration
7  server.port=8090
```

9.2. Instalación y puesta en marcha

El proceso de instalación y arranque de la aplicación es sencillo gracias a las capacidades de autoconfiguración de Spring Boot. Los pasos necesarios son los siguientes.

Paso 1 — Instalar Java y MySQL. Es necesario tener instalado Java 17 o superior, ya que es la versión mínima requerida por Spring Boot 3. También debe estar disponible una instancia de MySQL en ejecución.

Paso 2 — Crear la base de datos. Se debe crear en MySQL una base de datos vacía con el nombre configurado en el proyecto, así como un usuario con los permisos necesarios para acceder a ella.

Paso 3 — Configurar el archivo application.properties. En este archivo se especifican las credenciales de acceso a la base de datos, la URL de conexión y el comportamiento de Hibernate respecto a la generación automática de tablas. Gracias a la propiedad spring.jpa.hibernate.ddl-auto, Spring Boot puede crear o actualizar las tablas automáticamente al arrancar la aplicación, sin necesidad de importar ningún script SQL manualmente.

Paso 4 — Importar el proyecto en el IDE. Se abre el proyecto en Eclipse y se verifica que Maven descargue correctamente todas las dependencias declaradas en el pom.xml mediante el comando mvn clean install.

Paso 5 — Ejecutar la aplicación. La aplicación se inicia desde el IDE o mediante el comando mvn spring-boot:run en la terminal. Spring Boot arrancará el servidor Tomcat embebido y establecerá la conexión con la base de datos de forma automática.

Ilustración 9 - Código del main

```java
*PracticasApplication.java ×
package main;
import main.model.Usuario;

@SpringBootApplication
public class PracticasApplication {

    public static void main(String[] args) {
        SpringApplication.run(PracticasApplication.class, args);
    }

    @Bean
    public CommandLineRunner init(UsuarioRepository repo) {
        return args -> {

            // ADMIN
            Usuario admin = new Usuario();
            admin.setNombre("Admin");
            admin.setEmail("admin@admin.com");
            admin.setPassword("{noop}1234");
            admin.setRol("ROLE_ADMIN");

            repo.save(admin);

            // ALUMNO
            Usuario alumno = new Usuario();
            alumno.setNombre("Alumno");
            alumno.setEmail("alumno@alumno.com");
            alumno.setPassword("{noop}1234");
            alumno.setRol("ROLE_ALUMNO");

            repo.save(alumno);
        };
```

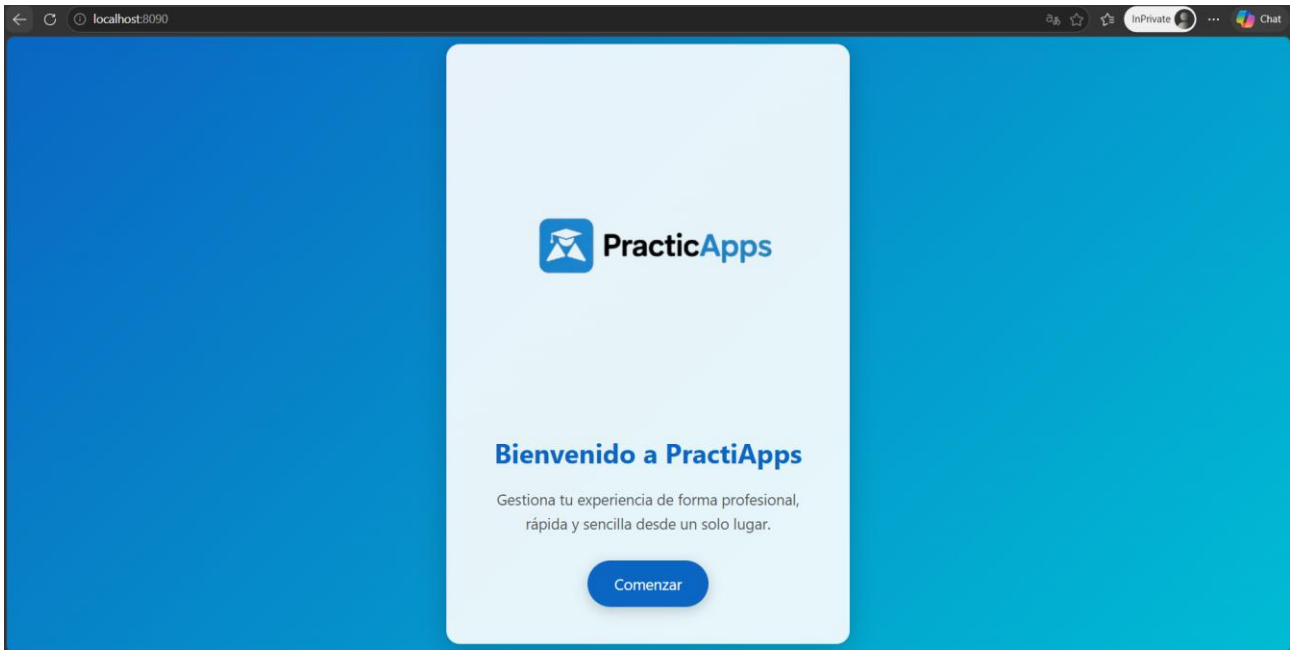
Ilustración 10 - Proyecto siendo desplegado

```
2026-04-18T17:54:16.659+02:00  INFO 22196 --- [  restartedMain] main.PracticasApplication                : No active profile set, falling b
2026-04-18T17:54:16.784+02:00  INFO 22196 --- [  restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : Devtools property defaults activ
2026-04-18T17:54:16.785+02:00  INFO 22196 --- [  restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For additional web related loggi
2026-04-18T17:54:18.463+02:00  INFO 22196 --- [  restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA re
2026-04-18T17:54:18.626+02:00  INFO 22196 --- [  restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository
2026-04-18T17:54:20.263+02:00  INFO 22196 --- [  restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port 809
2026-04-18T17:54:20.303+02:00  INFO 22196 --- [  restartedMain] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-04-18T17:54:20.304+02:00  INFO 22196 --- [  restartedMain] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache
2026-04-18T17:54:20.425+02:00  INFO 22196 --- [  restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded Web
2026-04-18T17:54:20.442+02:00  INFO 22196 --- [  restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: init
2026-04-18T17:54:20.522+02:00  INFO 22196 --- [  restartedMain] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Starting...
2026-04-18T17:54:20.926+02:00  INFO 22196 --- [  restartedMain] com.zaxxer.hikari.pool.HikariPool        : HikariPool-1 - Added connection
2026-04-18T17:54:20.929+02:00  INFO 22196 --- [  restartedMain] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Start completed.
2026-04-18T17:54:20.957+02:00  INFO 22196 --- [  restartedMain] o.s.b.a.h2.H2ConsoleAutoConfiguration    : H2 console available at '/h2-con
2026-04-18T17:54:21.354+02:00  INFO 22196 --- [  restartedMain] o.hibernate.jpa.internal.util.LogHelper  : HHH000204: Processing Persistenc
2026-04-18T17:54:21.592+02:00  INFO 22196 --- [  restartedMain] org.hibernate.Version                    : HHH000412: Hibernate ORM core ve
2026-04-18T17:54:21.688+02:00  INFO 22196 --- [  restartedMain] o.h.c.internal.RegionFactoryInitiator    : HHH000026: Second-level cache di
2026-04-18T17:54:22.219+02:00  INFO 22196 --- [  restartedMain] o.s.o.j.p.SpringPersistenceUnitInfo      : No LoadTimeWeaver setup: ignorin
2026-04-18T17:54:22.353+02:00  WARN 22196 --- [  restartedMain] org.hibernate.orm.deprecation            : HHH90000025: H2Dialect does not
2026-04-18T17:54:23.932+02:00  INFO 22196 --- [  restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator       : HHH000489: No JTA platform avail
2026-04-18T17:54:24.050+02:00  INFO 22196 --- [  restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFac
2026-04-18T17:54:24.651+02:00  WARN 22196 --- [  restartedMain] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabl
2026-04-18T17:54:24.944+02:00  INFO 22196 --- [  restartedMain] o.s.b.a.w.s.WelcomePageHandlerMapping    : Adding welcome page template: in
2026-04-18T17:54:25.400+02:00  INFO 22196 --- [  restartedMain] o.s.s.web.DefaultSecurityFilterChain     : Will secure any request with [or
2026-04-18T17:54:26.099+02:00  INFO 22196 --- [  restartedMain] o.s.b.d.a.OptionalLiveReloadServer       : LiveReload server is running on
2026-04-18T17:54:26.178+02:00  INFO 22196 --- [  restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8090 (htt
2026-04-18T17:54:26.199+02:00  INFO 22196 --- [  restartedMain] main.PracticasApplication                : Started PracticasApplication in
2026-04-18T17:54:39.278+02:00  INFO 22196 --- [nio-8090-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherSe
2026-04-18T17:54:39.279+02:00  INFO 22196 --- [nio-8090-exec-1] o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcher
2026-04-18T17:54:39.281+02:00  INFO 22196 --- [nio-8090-exec-1] o.s.web.servlet.DispatcherServlet        : Completed initialization in 2 ms
```

Paso 6 — Acceder desde el navegador. Una vez iniciada, la aplicación es accesible en http://localhost:8090. Se puede verificar el correcto funcionamiento iniciando sesión con las credenciales de prueba configuradas en el sistema.

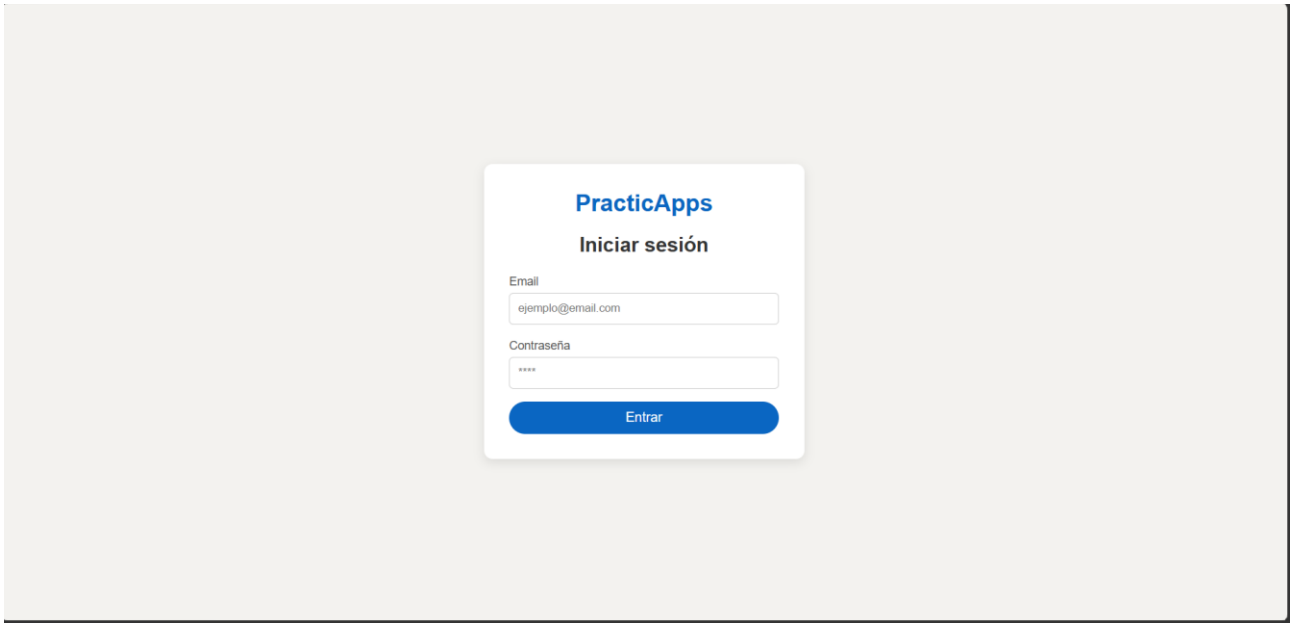
La pantalla de inicio actúa como punto de entrada a PracticApps. Presenta una tarjeta centrada sobre un fondo con degradado en tonos azules, mostrando el logotipo, un mensaje de bienvenida, una breve descripción del sistema y un botón "Comenzar" que guía al usuario hacia la autenticación.

Ilustración 11 - Aplicación desplegada



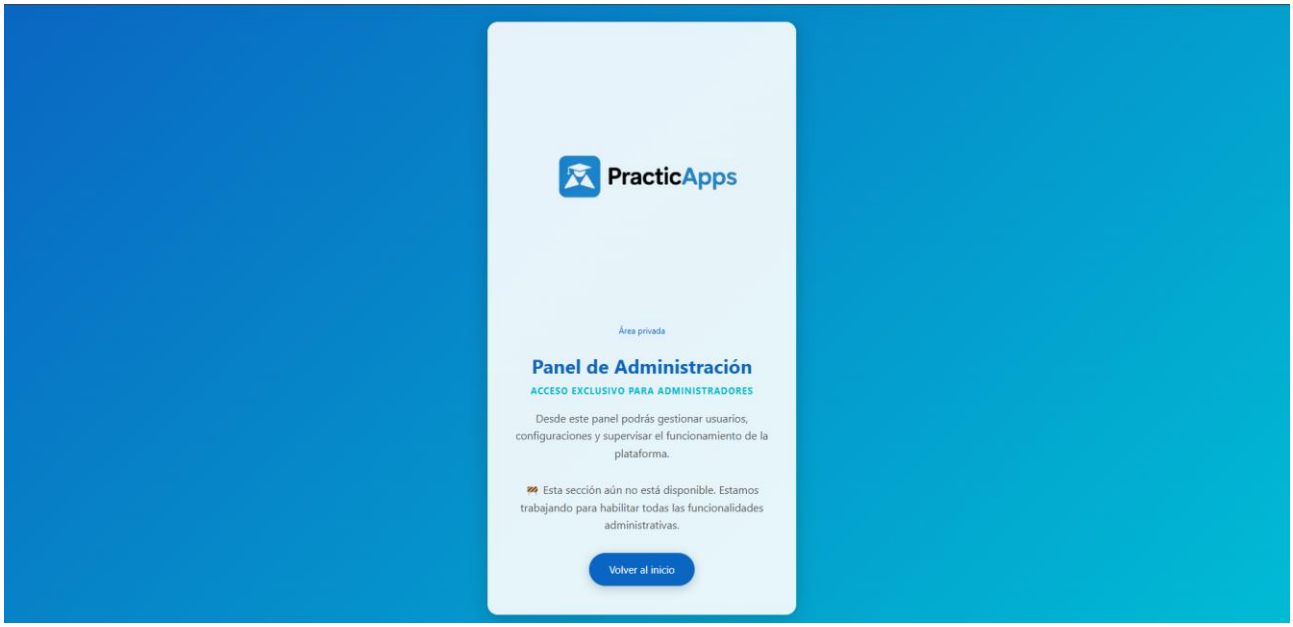
Tras pulsar "Comenzar", el usuario accede a la pantalla de login. Mantiene la misma línea visual con una tarjeta centrada que incluye el nombre de la aplicación, el título "Iniciar sesión" y un formulario con los campos de correo electrónico y contraseña. El botón "Entrar" valida las credenciales y redirige al usuario al panel correspondiente a su rol.

Ilustración 12 - Aplicación desplegada Login



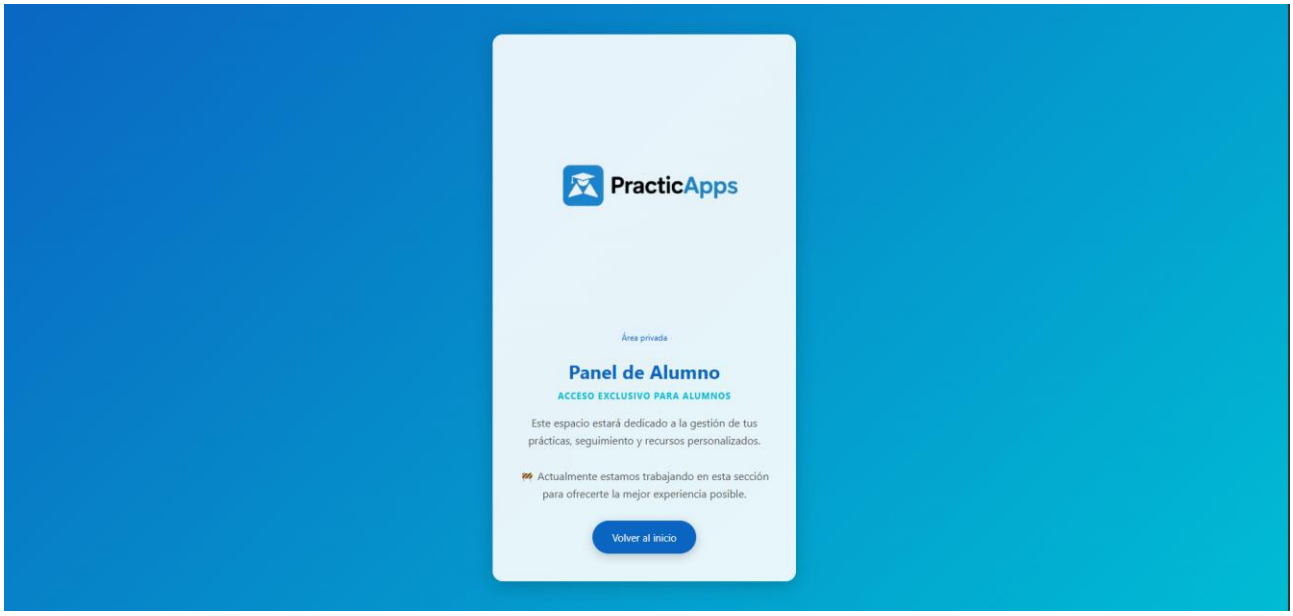
El Panel de Administración es accesible exclusivamente para usuarios con rol de administrador. Presenta la misma estructura visual que el resto de la aplicación, con el logotipo, el título del panel y una descripción de las funcionalidades previstas orientadas a la gestión de usuarios, configuraciones y supervisión del sistema. En esta fase del desarrollo el módulo muestra un aviso indicando que aún no está completamente disponible, junto con un botón "Volver al inicio".incluye un botón “Volver al inicio” para regresar a la pantalla principal de la aplicación.

Ilustración 13 - Aplicación panel administrador



El Panel de Alumno sigue la misma estructura visual que el resto de la aplicación. Muestra el logotipo, el título del panel y una descripción orientada a la gestión de prácticas y recursos del estudiante. Al igual que el módulo anterior, incluye un aviso de desarrollo en curso y el botón "Volver al inicio".

Ilustración 14 - Aplicación Panel alumno



10. GUÍA DE ESTILOS Y MANUAL DE USUARIO

Esta sección define la identidad visual de la aplicación y proporciona un manual para el uso eficiente del sistema. Su objetivo es garantizar consistencia, facilidad de uso y una experiencia uniforme para todos los usuarios.

10.1. GUÍA DE ESTILOS

- **Identidad del proyecto: PracticApps**

El nombre combina "prácticas" y "apps", reflejando la finalidad de la herramienta: gestionar prácticas formativas mediante una aplicación web.

- Corto, moderno y fácil de recordar.
- Diseñado para crear identidad de marca y reconocimiento inmediato.
- Logotipo e identidad visual
- El logotipo representa un estudiante con birrete, minimalista y reconocible.
- Tipografía:
- "Practic" en negro → seriedad, contraste y estabilidad.
- "Apps" en azul → tecnología, confianza y modernidad.
- Color principal azul → profesionalidad, coherencia y confianza visual.
- Diseño minimalista → escalable y adaptable a distintos formatos.
- Motivación del proyecto

PracticApps surge para optimizar la gestión de prácticas, un proceso tradicionalmente fragmentado y dependiente de métodos manuales o múltiples herramientas.

Beneficios:

- Asignación de prácticas a estudiantes.
- Registro y validación de horas.
- Evaluación del alumnado.
- Gestión documental centralizada.
- Comunicación fluida entre todos los actores.
- Diseño visual general
- Interfaz limpia, profesional y clara.
- Colores neutros + azul corporativo → buena legibilidad y coherencia visual.
- Botones visibles y formularios accesibles con validaciones claras.
- Diseño responsive → accesible en móviles, tablets y ordenadores.
- Importancia del diseño

Una interfaz bien diseñada reduce errores y mejora la experiencia del usuario, permitiendo que cualquier persona use la aplicación sin necesidad de formación previa. PracticApps prioriza la usabilidad sobre la complejidad visual.

Diseño visual general

- La interfaz de la aplicación ha sido diseñada siguiendo un enfoque limpio y profesional, evitando la sobrecarga visual y priorizando la claridad.
- Se han aplicado los siguientes principios:
- Diseño limpio y profesional: interfaz sencilla que facilita la comprensión.
- Colores neutros y azul corporativo: mejoran la legibilidad y refuerzan la identidad visual.
- Botones claros y visibles: facilitan la interacción del usuario.
- Formularios accesibles: estructurados de forma lógica, con validaciones y mensajes de error claros.
- Diseño responsive: adaptación a dispositivos móviles, tablets y ordenadores.

• Importancia del diseño

Una buena interfaz mejora significativamente la experiencia del usuario, reduce errores en el uso del sistema y permite que cualquier usuario pueda utilizar la aplicación sin necesidad de formación previa.

El diseño de PracticApps ha sido concebido con una filosofía de simplicidad y eficiencia, priorizando la usabilidad frente a la complejidad visual.

Paleta de Colores

La paleta de colores de la aplicación se ha diseñado para transmitir profesionalidad, confianza y claridad. Se ha optado por una combinación de azul corporativo como color primario, complementado con tonos neutros y un color de acento para las llamadas a la acción.

Los colores principales son los siguientes: el color primario es el azul #1A73E8, utilizado en la barra de navegación, botones principales y elementos destacados. El color secundario es el gris oscuro #343A40, utilizado en textos principales y fondos secundarios. El color de acento es el verde #28A745, utilizado en indicadores de estado positivo, botones de confirmación y mensajes de éxito. El color de alerta es el rojo #DC3545, utilizado para errores, estados de rechazo y advertencias. El color de fondo es el gris muy claro #F8F9FA, utilizado como fondo general de la aplicación.

Tipografía

La tipografía principal de la aplicación es **Segoe UI**, una fuente sans-serif de Google Fonts que ofrece excelente legibilidad en pantalla. Se utiliza en tres pesos: Regular (400) para el texto general, Medium (500) para subtítulos y etiquetas, y Bold (700) para títulos y elementos destacados.

El tamaño base de la fuente es 16px, con una escala tipográfica que sigue la proporción áurea: 10px para textos muy pequeños (metadatos), 12px para textos secundarios, 14px para textos de interfaz, 16px para el cuerpo del texto, 20px para subtítulos, 24px para títulos de sección y 32px para el título principal.

Iconografía

Se ha utilizado Font Awesome 6 como biblioteca de iconos. Los iconos se usan de forma consistente en toda la aplicación: el icono de reloj para las horas, el icono de archivo para los documentos, el icono de usuario para los perfiles, el icono de gráfico de barras para los informes y el icono de campana para las notificaciones.

10.2. MANUAL DE USUARIO

Usuario general

Acceso a la aplicación:

Para acceder a la plataforma, abre tu navegador web (se recomienda Google Chrome, Mozilla Firefox o Microsoft Edge en sus versiones más recientes) e introduce la dirección URL de la aplicación. Verás la página de inicio de la plataforma.

Inicio de sesión:

Haz clic en el botón "Iniciar sesión" de la barra de navegación. Introduce tu correo electrónico y contraseña en el formulario y haz clic en "Entrar". Si las credenciales son correctas, serás redirigido a tu panel de control personalizado.

Manual del alumno

Opciones:

- Iniciar sesión en la plataforma.

- Consultar prácticas asignadas.
- Registrar horas realizadas.
- Subir documentación requerida.
- Consultar evaluaciones.

Panel de control:

Al iniciar sesión, el alumno accede a su panel de control, donde puede ver un resumen de su situación actual: horas registradas vs. horas previstas, últimos registros pendientes de validación, notificaciones recientes y accesos rápidos a las funciones más utilizadas.

Registro de horas:

Para registrar las horas de un día, accede a la sección "Mis horas" desde el menú lateral. Haz clic en "Nuevo registro", selecciona la fecha, introduce el número de horas trabajadas y describe brevemente las actividades realizadas. Haz clic en "Guardar" para enviar el registro. El registro quedará en estado "Pendiente" hasta que tu tutor de empresa lo valide.

Gestión de documentos:

Desde la sección "Documentos", puedes subir los archivos requeridos para tus prácticas. Haz clic en "Subir documento", selecciona el tipo de documento, adjunta el archivo desde tu dispositivo y haz clic en "Subir". Los formatos aceptados son PDF, DOCX, XLSX e imágenes (JPG, PNG).

Consulta de evaluaciones:
En la sección "Evaluaciones", el alumno puede consultar las valoraciones realizadas por el tutor de empresa y el tutor de centro, junto con los comentarios asociados.

Opciones:

- Iniciar sesión en la plataforma.

- Supervisar el desarrollo de las prácticas.
- Revisar evaluaciones.
- Generar informes académicos.

Seguimiento de alumnos:

El tutor de centro tiene acceso a un listado de todos sus alumnos asignados con su estado actual. Puede hacer clic en cualquier alumno para ver el detalle de sus prácticas: horas registradas, documentos subidos, evaluaciones previas y comunicaciones.

Evaluación de alumnos:

Para registrar una evaluación, accede al perfil del alumno y haz clic en "Nueva evaluación". Rellena los criterios de evaluación con las puntuaciones correspondientes, añade comentarios si lo consideras oportuno y guarda la evaluación. El alumno recibirá una notificación automática.

Generación de informes:
Desde la sección "Informes", el tutor de centro puede generar informes académicos sobre el progreso del alumno, incluyendo horas realizadas, evaluaciones y estado general de las prácticas. Estos informes pueden descargarse en formato PDF.

Manual del Administrador

Administrador

- Iniciar sesión en la plataforma.

- Gestionar usuarios y roles.
- Administrar empresas y convenios.
- Asignar prácticas a alumnos.
- Configurar el sistema.

Gestión de usuarios:

El administrador puede acceder al panel de gestión de usuarios desde el menú de administración. Desde allí puede ver todos los usuarios registrados, crear nuevos usuarios, editar sus datos, cambiar su rol o desactivar su cuenta.

Asignación de prácticas:

Para asignar un alumno a unas prácticas, accede a "Gestión de prácticas" y haz clic en "Nueva asignación". Selecciona el alumno, la empresa, el tutor de empresa y el tutor de centro. Establece las fechas de inicio y fin y el número de horas previstas. Confirma la asignación y todos los implicados recibirán una notificación.

Asignación de prácticas:
Para asignar un alumno a unas prácticas, accede a "Gestión de prácticas" y haz clic en "Nueva asignación". Selecciona el alumno, la empresa, el tutor de empresa y el tutor de centro. Establece las fechas de inicio y fin y el número de horas previstas. Confirma la asignación y todos los implicados recibirán una notificación.

Configuración del sistema:
Desde la sección de configuración, el administrador puede definir parámetros generales como tipos de documentos, criterios de evaluación, notificaciones automáticas y permisos de usuario.

Tutor de empresa

- Iniciar sesión en la plataforma.

- Validar las horas registradas por los alumnos.
- Evaluar el desempeño del alumno.
- Subir informes relacionados con la empresa.

Validación de horas:
El tutor de empresa puede revisar y validar las horas registradas por el alumno accediendo a la sección "Horas de alumnos". Allí encontrará un listado de los registros pendientes. Para cada entrada, puede revisar la información proporcionada y hacer clic en "Validar" si es correcta o "Rechazar" si requiere corrección. En caso de rechazo, es recomendable añadir un comentario explicativo para que el alumno pueda realizar las modificaciones necesarias.

Evaluación del alumno:
Para evaluar el desempeño del alumno, accede a su perfil y selecciona la opción "Evaluar alumno". Completa los criterios establecidos, añade observaciones si lo consideras oportuno y guarda la evaluación. Esta evaluación será visible tanto para el alumno como para el tutor de centro.

Gestión de informes:
Desde la sección "Informes", el tutor de empresa puede subir documentos relacionados con el seguimiento del alumno, como informes de progreso o evaluaciones internas. Haz clic en "Subir informe", selecciona el archivo y confirma la acción. Los formatos permitidos son PDF, DOCX y XLSX.

Este manual facilita el aprendizaje del sistema y permite que cualquier usuario pueda utilizar la aplicación de forma autónoma.

11. WEBGRAFÍA Y FUENTES DE INFORMACIÓN

La webgrafía recoge el conjunto de fuentes de información utilizadas durante el desarrollo del proyecto. Incluye documentación oficial, tutoriales, foros técnicos y recursos online que han servido de apoyo para la implementación de funcionalidades y la resolución de problemas.

- Este apartado es importante porque:
- Permite verificar la información utilizada.
- Aporta credibilidad al proyecto.
- Facilita futuras consultas o ampliaciones.

Entre las principales fuentes utilizadas destacan:

- La documentación oficial de Spring Boot disponible en la web de Spring (https://spring.io), la cual ha sido fundamental para comprender la estructura del framework, el uso de dependencias y el desarrollo de aplicaciones web.
- La herramienta Spring Initializr (https://start.spring.io), utilizada para la generación inicial del proyecto y la configuración de sus dependencias.
- Tutoriales y guías prácticas sobre desarrollo con Spring Boot, como los disponibles en OpenWebinars (Soluciones de aprendizaje para empresas | OpenWebinars) y otras plataformas educativas, que han facilitado la comprensión de conceptos como la creación de APIs REST, la conexión con bases de datos y la estructura en capas.
- Recursos relacionados con la persistencia de datos mediante Spring Data JPA, utilizados para la gestión de la base de datos y la implementación de operaciones CRUD.
- Documentación y ejemplos sobre seguridad en aplicaciones web con Spring Security, empleados para la implementación de sistemas de autenticación y control de acceso.

Además, es importante destacar que el desarrollo de este proyecto también se ha basado en los conocimientos adquiridos durante el ciclo formativo, especialmente en los módulos de desarrollo web, bases de datos y entornos de desarrollo, los cuales han sido fundamentales para la correcta realización de la aplicación.